Module 1 Report: Core Renderers

Exam Number: B252751

October 13, 2026

1 Introduction and AI Usage Declaration

This report documents the implementation of the Whitted-Style Ray Tracer (WSRT) and Distributed Ray Tracer (DRT) for Module 1.

AI Usage Declaration: I designed the overall architecture and provided the ideas, report structure and original text. For implementation, I provided mathematical formulas and requirements to an AI coding assistant, which generated code. Some methods came from Lab 1 and the lectures; others were suggested by AI and then it study and applied by me. My development process was to checking individual components on small examples before combining them. AI assistance helped code generation and debugging for the integrators, reflection/refraction, BRDF, camera, mesh loading and parser, as well as scene assets, tests and image annotations.

Scene and evidence overview: I took the reference photograph in the university's Teviot building. The simple scenes isolate the core features; the creative scenes combine them in a cafe table composition based on this photograph. The open laptop, pale marble tabletop with a dark wooden border, metal condiment rack, bottles and green chairs provide the main correspondences. A polished tray was added beneath the rack to make recursive reflections easier to inspect. The screen, keyboard, marble and label textures are procedural assets rather than image crops from the photograph. The reconstruction approximates the layout and materials, rather than matching a calibrated camera or reproducing the photograph pixel by pixel.

The lettered regions in the four figures identify visual examples. Numerical checks are used for properties that an image cannot establish on its own, such as TIR and BRDF energy bounds. WSRT uses a single pixel-centre ray; DRT averages independent lens and light samples at a fixed pixel centre. The current DRT evaluates direct illumination and recursive mirror/dielectric paths, without diffuse indirect illumination.

1.1 Architecture and Development Process

I first decided the overall file structure and the responsibilities of the components. The parser reads the scene description and constructs the in-memory `Scene`, which holds the camera, film settings, geometry, materials and lights. `mesh.cpp` loads OBJ data. The pixel loops belong to the concrete integrators, `whitted.cpp` and `distributed.cpp`; they call `Camera::GenerateRay` to map a two-dimensional pixel sample to a three-dimensional viewing ray. My understanding of this mapping drew on the mathematics from Lab 1 and the lectures [1, Part I, pp. 1–2].

The ray is tested against the geometry to find the nearest intersection. The integrator uses its position, normal and material to evaluate illumination [2]. For each light, a shadow ray tests whether another object blocks the segment between the surface and the light. An occluded point receives no direct contribution from that light. For a visible light on the front side, diffuse shading uses the surface-normal/light-direction cosine, and WSRT uses the halfway vector for Blinn–Phong highlights. Mirror materials instead launch recursive reflected rays. Finally, `Image` stores the pixels and the image output code writes the result. The common `integrator.cpp` contains the scene-wide intersection and occlusion queries and the integrator factory. I chose

to share components between WSRT and DRT to simplify debugging and keep their behaviour consistent.

Formula-guided Code Generation: I provided formulas and implementation requirements to AI rather than writing every implementation by hand. Examples included pixel-centre sampling, the camera basis and field-of-view mapping, the ray–triangle intersection equation

$$\mathbf{O} + t\mathbf{D} = \mathbf{V}_0 + u\mathbf{E}_1 + v\mathbf{E}_2,$$

and barycentric normal interpolation,

$$\mathbf{n} = \text{normalize}\big((1-u-v)\mathbf{n}_0 + u\mathbf{n}_1 + v\mathbf{n}_2\big).$$

The triangle equation solves for t, u and v using cross products and scalar triple products. I checked components using small examples before integrating them. These are examples of the mathematical input I supplied, rather than verbatim quotations of the original prompts. A specific visual feedback and the division of work are recorded in the reflection section below.

2 WSRT (Whitted-Style Ray Tracer)

2.1 Specular Reflection

Design Decisions: Recursive reflection uses $\mathbf{d}_r = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n}_g)\mathbf{n}_g$, where $\mathbf{d}$ is the incident direction and $\mathbf{n}_g$ is the geometric normal [3, Ch. 8, p. 26]. The maximum depth is configurable (MD=8 for simple, MD=10 for creative), bounding the work for repeated mirror and dielectric encounters. At each bounce the path throughput is multiplied component-wise by the material reflectance; reflectance below one attenuates successive reflections. A path reaching the depth limit contributes zero radiance.

Difficulties Encountered: The implementation originally mixed incident and outward direction conventions: `Reflect` expects $\mathbf{w}_o = -\mathbf{d}$. Passing $\mathbf{d}$ instead produced the wrong reflected direction. Another defect returned throughput at the depth limit, treating a path weight as emitted light. The fixes use the outward convention and return zero at termination. Analytic checks verify preservation of the tangential direction component, reversal of the normal component, and zero termination contribution. **Example Render:**

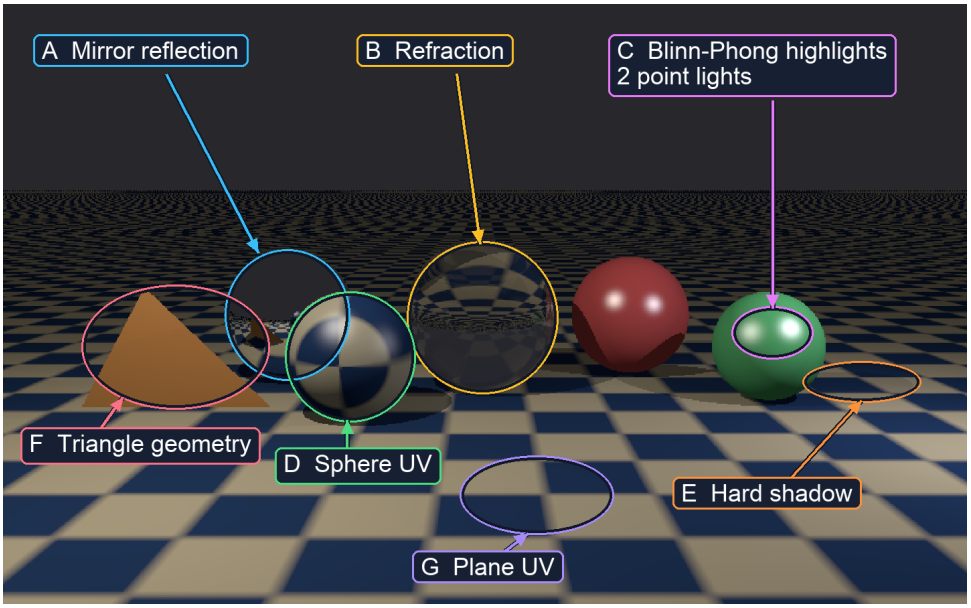


Figure 1: WSRT_simple: A, mirror-sphere reflection; B, glass-sphere refraction; C, green-sphere Blinn–Phong highlights; D, spherical checker UVs; E, hard ground shadow; F, individual orange triangles; G, planar checker UVs. Original render: 900×560, one primary sample per pixel, MD=8.

Observations: Region A in Figure 1 contains a compressed reflection of the checkerboard floor and the nearby orange geometry. In Figure 2, region A instead shows inverted bottle bodies, label bands and rack elements in the flat tray. The tray reflectance is $(0.91, 0.92, 0.93)$, compared with $(0.95, 0.96, 0.97)$ for the simple mirror sphere. These are recursive scene reflections, distinct from the small highlights produced by local shading.

Scene Iteration and My Contribution: In an earlier version of `WSRT_creative.pbrt`, the mirror surfaces were mainly the thin rods and rings of the bottle rack. I noticed that the mirror effect was not sufficiently clear and raised this as visual feedback. The parts occupied few pixels, making recognizable reflections difficult to see. The diagnosis focused on the small visible mirror area and the limited contrast of reflected objects. This was a scene-presentation issue; this iteration did not involve my changing the reflection formula. My feedback is described here from recollection, not quoted as an exact original prompt.

Following my feedback, AI implemented a polished tray below the rack. Its reflective surface consists of `polishedTrayA` and `polishedTrayB`, covering 3.20×2.84 scene units. The local triangles are rotated by $-90°$ about the x axis to form a horizontal surface at $y = 0.115$, with mirror reflectance $(0.91, 0.92, 0.93)$. The camera remained at $(4.4, 4.2, 8.2)$, looking towards $(-0.1, 0.9, -0.05)$ with up vector $(0, 1, 0)$ and FOV $46°$; the main café layout was retained.

The qualitative expectation was that a broad, suitably positioned mirror would show structured reflections of the bottles, labels and rack. After re-rendering, I checked the image for these features. They were clearly visible in the tray and became region A in Figure 2. No per-pixel reference colours or complete ray paths were recorded for this iteration. The result demonstrates a clearer presentation of recursive reflection, rather than proving the reflection formula for every input.

2.2 Specular Refraction (Fresnel and TIR)

Design Decisions: Snell's law, $\eta_i \sin\theta_i = \eta_t \sin\theta_t$, determines the transmitted direction [3, Ch. 9, pp. 29–30], while the dielectric Fresnel equations assign weights R and $1 - R$ to reflection and transmission. The entry test $\mathbf{n}_g \cdot \mathbf{d} < 0$ is performed before orienting the normal, so

an exiting ray selects glass-to-air indices. Transmission throughput includes $(\eta_i/\eta_t)^2$ for the radiance convention. If $\sin^2\theta_t \geq 1$, no transmitted direction is generated and $R = 1$. For air-to-glass incidence, reflectance rises towards grazing angles; at normal incidence with $\eta = 1.5$, $R = 0.04$.

Difficulties Encountered: Incorrect direction signs and premature normal flipping caused errors in exit-medium selection. The DRT also discarded the weaker Fresnel branch; both nonzero branches are now evaluated. Numerical checks cover air-to-glass incidence at 30° ($\sin\theta_t = 1/3$), normal-incidence reflectance of 4%, and glass-to-air incidence at 60°, which exceeds the critical angle and gives TIR. **Example Render:**

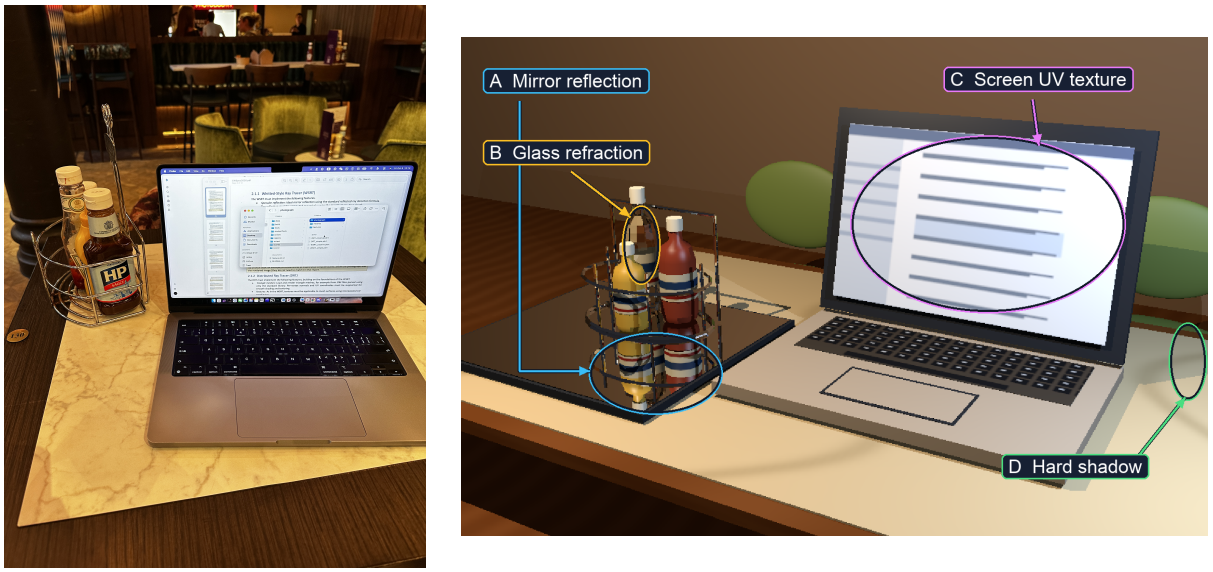


Figure 2: My reference photograph taken in the university's Teviot building (left) and WSRT_creative reconstruction (right). A marks bottle and rack reflections in the added polished tray; B identifies the transparent amber bottle ($\eta = 1.5$); C marks the screen texture; D marks the hard tabletop shadow. Original render: 960×640, one primary sample per pixel, MD=10, two point lights.

Observations: Region B in Figure 1 is the clearest refraction example: the checker pattern seen through the central glass sphere ($\eta = 1.52$) is displaced and curved relative to the surrounding floor. Region B in Figure 2 identifies the narrower amber bottle ($\eta = 1.5$), whose body combines transmitted scene detail and surface reflection. Occlusion by other bottles, the rack and the opaque label makes this a less isolated optical example. The bottle is a closed solid surface of revolution, not a hollow bottle with liquid or a glass slab; its tint is a surface weight rather than volumetric absorption. The labels are separate opaque surfaces. Neither the dark portions of the sphere nor the bottle annotation establish a particular TIR event without a pixel-path trace; TIR is supported by the numerical test above.

2.3 Blinn-Phong Shading

Design Decisions: Ambient is baseColor × ambientScale. Diffuse uses baseColor$/\pi \times \max(0, \mathbf{n} \cdot \mathbf{w}_i) \times I/r^2$. Specular uses the half-vector $\mathbf{h} = \text{normalize}(\mathbf{w}_o + \mathbf{w}_i)$ and $k_s \max(0, \mathbf{n} \cdot \mathbf{h})^{\text{shininess}}$, multiplied by I/r^2. The half-vector bisects the outward viewing and light directions [1, Part III, Sec. 3, p. 4]. Increasing the exponent concentrates the lobe around alignment with the normal. **Example Render:** Region C in Figure 1 circles two bright highlights on the green sphere (shininess=60); the red sphere (shininess=120) also has two visible highlights. The scene file specifies two point lights. These localized bright spots differ from the structured scene reflection

in the mirror sphere. The red and green spheres also differ in position, colour and specular weight, so their appearance is not a controlled comparison of shininess alone.

2.4 Point Light Sources and Shadows

Design Decisions: Point lights use intensity I and distance attenuation I/r^2. Shadow rays test the finite segment towards each light, with maximum distance $\text{dist} - 2\epsilon$, so an object beyond the light cannot block it. The origin is offset by $\mathbf{p} + \epsilon \mathbf{n}_{g,\text{oriented}}$, where $\epsilon = 10^{-4} \max(1, \text{dist})$ and the geometric normal is oriented towards the light-facing side. This avoids numerical self-intersection. Mirror and refraction rays instead use a small offset along their outgoing direction. **Example Render:** Region E in Figure 1 shows a sharply bounded ground shadow to the right of the green sphere. Region D in Figure 2 marks the tabletop shadow beside the laptop; the rack also casts narrow, angular shadows. Each ideal point light gives binary visibility at a surface point, and the two lights can produce overlapping shadows with different brightness levels. The light positions and intensities are established by the scene files, since the point lights are not drawn as visible panels.

2.5 Textures and UV Mapping

Design Decisions: Spheres use $u = \phi/(2\pi)$, with longitude derived from $\text{atan2}(n_y, n_x)$ and wrapped into the texture domain, and $v = \arccos(n_z)/\pi$. Planes use local UVs; triangle UVs are interpolated with barycentric weights. PPM textures (P3/P6) are converted from sRGB to linear colour when loaded and sampled bilinearly, with UV scaling and wrapping. **Example Render:** Region D in Figure 1 shows the 32×32 blue/beige checker texture wrapping over an analytic sphere. Region G shows the same asset repeated on the rotated ground plane, with u and v scales of 0.55. The broad checks make the coordinate mapping visible, although the distant floor still aliases under single-sample rendering. Region C in Figure 2 shows a procedural 128×80 screen image mapped across two triangles; its lines remain aligned across the panel rather than being assigned a constant material colour.

2.6 Geometry and Transformations

Design Decisions: Sphere intersections solve a quadratic, planes use local $z = 0$, and individual triangles use Möller–Trumbore intersection. The nearest positive hit determines visibility. Rays are transformed by the inverse object matrix and normals by the inverse transpose [1, Part IV, Sec. 3, p. 5]. Transformed ray directions are **not** re-normalized: retaining the same parameter in $\mathbf{p}(t) = \mathbf{o} + t\mathbf{d}$ makes hits comparable across differently scaled objects. **Example Render:** Region F in Figure 1 identifies the two individual orange triangles; A–D contain analytic spheres, and G is a plane rotated into the ground orientation. These give distinct geometry examples within one view. A numerical scaling check is stronger evidence for parameter consistency: a unit sphere scaled by two, viewed from $(0, 0, 5)$ along $-z$, is hit at $t = 3$ and world-space $z = 2$. Inverse-transpose checks also verify that a normal remains perpendicular to a transformed tangent under nonuniform scale.

3 DRT (Distributed Ray Tracer)

3.1 Triangle Meshes (OBJ)

Design Decisions: The OBJ loader supports `v/vt/vn` corner indices, negative indices and fan triangulation of polygons. When normals are absent, adjacent unnormalized face cross products are accumulated to produce area-weighted vertex normals. Mesh vertices are transformed into world space; supplied or generated normals use the inverse transpose. Interpolated vertex

normals smooth shading within the triangular surface, but do not change its geometric shapes. **Example Render:**

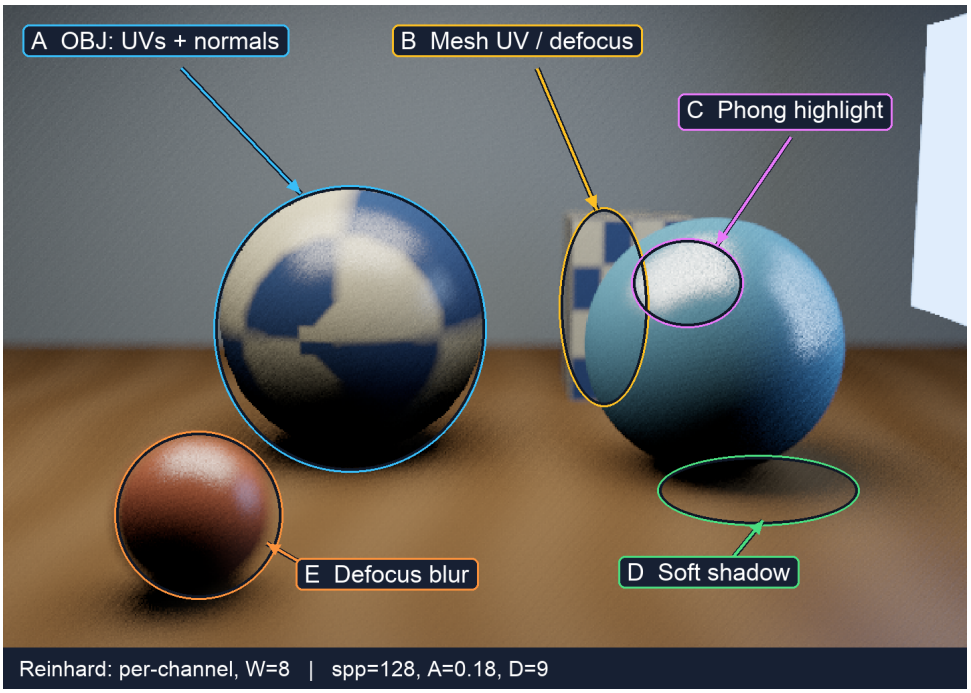


Figure 3: DRT_simple: A, textured OBJ geodesic sphere with vertex normals; B, background checker-box mesh; C, blue-sphere Phong highlight; D, soft shadow beneath and beside the blue sphere; E, foreground red sphere used to inspect defocus. Original render: 600×400, 128 lens/light samples per pixel, MD=4, aperture radius 0.18 and focus distance 9; per-channel extended Reinhard with $W = 8$.

Observations: Region A in Figure 3 is the OBJ asset `uv_geodesic.obj`, with 320 triangular faces, 960 texture-coordinate entries and 162 vertex normals. Its rounded shading is consistent with interpolated normals; it is not an analytic sphere. Region B is the visible portion of a separate 12-face OBJ box. The creative bottles, rack rings, rods and laptop box components provide further mesh examples. The loader tests check actual UV/normal corner references, since a rounded appearance alone cannot verify file loading or indexing.

3.2 Textures on Meshes

Design Decisions: UV coordinates are interpolated using the triangle-intersection barycentric weights: $\mathbf{uv} = b_0\mathbf{uv}_0 + b_1\mathbf{uv}_1 + b_2\mathbf{uv}_2$. The base colour is DiffuseColor $\times$ TextureAt($\mathbf{uv}$) and is passed into the BRDF. A texture declaration alone is insufficient: valid corner UV indices and use of the sampled colour in shading are both necessary. Normal interpolation controls lighting orientation, whereas UV interpolation selects positions in the texture. **Example Render:** The blue/beige pattern in region A of Figure 3 follows the curved mesh surface, while region B shows checks on a planar box face. In Figure 4, region C shows repeated key shapes on the laptop keyboard, mapped from the 128×64 procedural PPM across two triangles. The visible keys are texture detail rather than individually modelled keycaps. A separate check confirms that a black diffuse texel produces zero diffuse BRDF contribution.

3.3 Area Light Sources and Soft Shadows

Design Decisions: PCG32 pseudo-random samples select points uniformly over each rectangular emitter. The area density $1/A$ is converted to a solid-angle PDF $p_\omega = r^2/(A\cos\theta_{\text{light}})$.

Each visible sample contributes $\beta f \cos\theta_{\text{surface}} L_e / p_\omega$, averaged over samples per light and then over camera samples. The current scenes use one sample per light for each camera sample. Partial visibility of an extended emitter produces a penumbra: different sampled points can be visible or occluded at the same surface point [1, Part VI, Sec. 5, pp. 7–8].

Difficulties Encountered: The rectangle sampling frame originally mixed row and column coordinates, and the sampling normal was not transformed into world space. Checks of sampled corners, world-space normals, area and the solid-angle PDF address these defects independently of image appearance. **Example Render:**

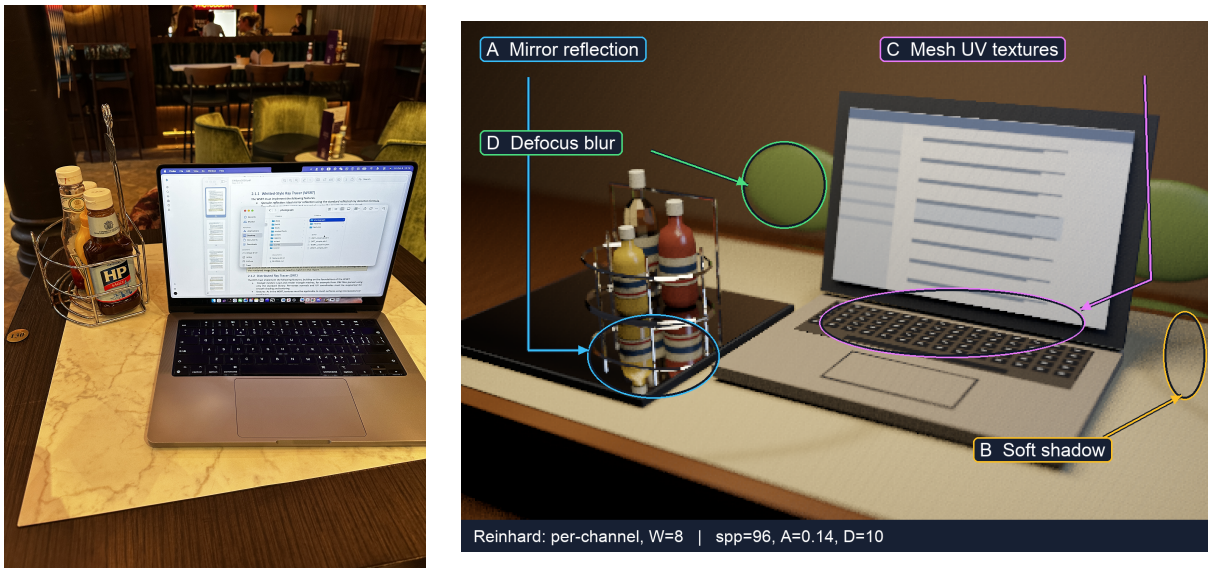


Figure 4: My reference photograph taken in the university's Teviot building (left) and DRT_creative reconstruction (right). A marks recursive bottle reflections in the added tray; B marks the tabletop penumbra beside the laptop; C marks the keyboard UV texture; D identifies the background green chair used to inspect defocus. Original render: 480×320, 96 lens/light samples per pixel, MD=5, aperture radius 0.14 and focus distance 10; per-channel extended Reinhard with $W = 8$.

Observations: Region D in Figure 3 shows a broad transition from the dark contact region below the blue sphere to the brighter floor. Region B in Figure 4 similarly marks a gradual tabletop shadow transition beside the laptop. The creative emitters have areas 29.4 and 9.0 in scene units, replacing the WSRT point lights. At 128 and 96 samples per pixel, respectively, visible grain remains across lit surfaces and shadow edges. The penumbra is the spatial change in mean illumination; the speckling is Monte Carlo error in its estimate. The bright panel at the right of DRT_simple is a visible emitter, not itself a soft shadow. The two creative images differ in material model, light geometry, resolution and camera sampling, so they illustrate the features rather than providing an integrator-only controlled comparison.

Higher-sample Renders: After increasing DRT_simple from 32 to 128 samples per pixel and DRT_creative from 24 to 96, I inspected both the original and annotated images. The sphere highlights, tabletop and background appeared smoother, and the soft shadows were easier to distinguish. A small amount of grain remained visible on enlargement.

3.4 Phong BRDF and Energy Conservation

Design Decisions: The BRDF is $k_d/\pi + k_s \frac{e+2}{2\pi} \max(0, \text{Reflect}(\mathbf{w}_i, \mathbf{n}) \cdot \mathbf{w}_o)^e$, where $k_s = \text{clamp}(\text{specular}, 0, 1)$ and $k_d = \text{clamp}(\text{texturedDiffuse}, 0, 1) \times (1 - k_s)$. The normalization factor controls the specular lobe's integrated contribution as e changes; the $(1 - k_s)$ factor separately

reserves a per-channel energy budget for diffuse reflection. The integrator multiplies the surface cosine once, outside the BRDF, and the BRDF is zero outside the supported hemisphere. **Example Render:** Region C in Figure 3 marks the broad upper-left highlight on the blue sphere, with exponent $e = 60$ and $k_s = (0.35, 0.35, 0.35)$. Region A in Figure 4 is instead an ideal mirror reflection, not a Phong highlight. The highlight provides a visual example of glossy response but does not prove energy conservation. The numerical check integrates $f \cos\theta$ over the hemisphere using a 256×256 grid for diffuse input 1, $k_s = 0.8$ and $e = 20$, at viewing angles $0°$, $45°$ and $80°$; the reflected energy is bounded by 1.0001 for these cases.

3.5 Defocus Blur and Depth of Field

Design Decisions: The thin-lens camera samples a uniform aperture disk using $r = A\sqrt{u}$ and $\theta = 2\pi v$, where A is the aperture radius. For a pinhole direction $\mathbf{d}$, the focal point is $\mathbf{p}_{\text{camera}} + t_f\mathbf{d}$, with $t_f = D/(\mathbf{d} \cdot \mathbf{forward})$. Each sampled lens origin is aimed at this common point on the plane at forward distance D [3, Ch. 11, pp. 37–39]. Objects away from this plane are projected to a spread of positions as the lens origin varies; $A = 0$ reduces the construction to a pinhole.

Difficulties Encountered: Advancing a fixed distance along every pinhole direction originally formed a spherical focus surface. Intersecting the planar focus surface fixes the off-axis behaviour. A numerical check verifies that rays for a noncentral pixel, generated from different lens samples, meet the same point on the $D = 10$ focal plane. **Example Render:** Figure 3 uses $A = 0.18$, $D = 9$ and 128 samples per pixel. The foreground red sphere in region E and the rear checker box in region B have softened boundaries, while the main checker mesh retains more recognizable pattern structure. Figure 4 uses $A = 0.14$, $D = 10$ and 96 samples per pixel; region D selects the green chair behind the screen as a background defocus example, relative to the laptop and bottle arrangement. Grain on a flat-coloured surface is not evidence of defocus. Since no same-scene $A = 0$ reference is included, these observations are consistent with the lens model but do not isolate blur from sampling and resolution effects quantitatively.

3.6 Reinhard Tone Mapping

Design Decisions: Both DRT scenes use extended Reinhard tone mapping per RGB channel, with white point $W = 8$: $c' = c(1+c/W^2)/(1+c)$. The sequence is HDR sample averaging, tone mapping, clamping to $[0, 1]$, and finally sRGB encoding when writing the image. Averaging before this nonlinear mapping avoids averaging already-compressed samples. Tone mapping compresses the radiance range; sRGB encoding converts display-linear values to the output encoding and is a separate operation. **Example Render:** Both DRT figures retain dark object regions alongside a bright screen or emitter, but this appearance is only qualitative context. The mapping sends $c = W$ to 1, and values above W can exceed 1 before the final clamp. The current logs report post-mapping, post-clamp film maxima of 0.970703 for DRT_simple and 1.000000 for DRT_creative, before sRGB encoding. The latter is consistent with channels reaching display white, but supplies neither a clipped-pixel count nor the original HDR values. The white screen or panel therefore cannot establish an absence of clipping. Formula checks independently verify white-point mapping ($c = W = 4$ gives 1) and the $W = 1$ identity case.

4 PBRT Scene Compatibility

The parser supports a PBRT subset: Film, Sampler, Integrator, LookAt, Camera, WorldBegin/End, attribute scopes, Material/NamedMaterial, PPM imagemap Texture, point LightSource, diffuse AreaLightSource, spheres, inline triangle meshes and Translate/Scale/Rotate. The project's named, brace-delimited scene format is also retained and supports OBJ asset loading. Asset paths are resolved relative to the scene file. Standard PBRT FOV uses the

smaller image axis; the project block format uses the larger axis, a distinction relevant when translating cameras.

Error Handling and Checks: The `broken.pbrt` fixture verify diagnostics for a missing texture, an unknown primitive and unsupported object syntax. The `standard.pbrt` fixture checks standard statements, scopes, multiline mesh arrays, named textures, transforms and point/area emitters. All four reported-scene logs record zero parse warnings and errors. These checks establish behaviour for the supported inputs, rather than full PBRT equivalence: includes, general instancing, animation and nested media are unsupported, and no PBRT-v4 differential rendering comparison has been performed. The current implementation also has no diffuse indirect lighting, so the laptop's emissive screen is not evidence of simulated screen illumination on the table.

Verification Summary: The existing `make check-module1` suite contains 50 checks covering optical directions and weights, transformed intersections and normals, mesh attributes, light sampling, BRDF energy, camera focus, tone mapping and parser diagnostics. These targeted checks complement the annotated images; they do not establish correctness for arbitrary scenes.

5 Profiling Results

Table 1 reports the existing render-stage log measurements on an Apple M2 Pro. The renderer is single-threaded and compiled with Apple clang, C++17 and `-O3`. Parsing and total timers did not produce complete stage measurements, so they are not included. Both updated DRT measurements were collected serially. The table records actual runs, not a uniformly controlled performance baseline.

Scene	Resolution	Samples/Pixel	Max Depth	Primitives	Render Time (s)
WSRT_simple	900x560	1	8	8	0.461692
WSRT_creative	960x640	1	10	1230	22.864423
DRT_simple	600x400	128	4	336	252.429880
DRT_creative	480x320	96	5	1230	437.498226

Table 1: Logged render-stage times for the current Module 1 images. Samples per pixel are primary rays for WSRT and lens/light samples at fixed pixel centres for DRT. Both DRT runs were serial.

The creative scenes expand to 1230 intersection primitive, include the bottle, rod and rack meshes and the two tray triangles. Their cost is affected by linear traversal of these primitives and recursive mirror/dielectric paths. DRT additionally averages 96 or 128 lens/light samples per pixel. Resolution, geometry, maximum depth, light type and materials vary across rows, so the timings cannot isolate any one feature's cost. No BVH is implemented in this module. The retained BF field does not introduce diffuse branching in the current integrator and should not be interpreted as four diffuse paths per bounce.

6 Conclusion

The simple images provide identifiable examples of mirror reflection, glass refraction, local highlights, shadows, UV textures and geometry. The updated creative reconstruction combines these features in a photograph-inspired laptop and source track scene. Its added tray makes recursive reflections especially clear. WSRT produces sharp point-light shadow boundaries, while DRT adds area-light penumbras, mesh textures and thin-lens sampling; Monte Carlo grain remains visible at the reported sample counts.

The main implementation difficulties were consistent incident/outward direction conventions, correct dielectric entry and exit handling, and preservation of intersection parameters and normals under transformations. Numerical tests supply evidence where visual inspection is insufficient, particularly for TIR, energy bounds and tone-mapping formulas. Useful next steps are a same-scene pinhole reference for defocus, uniformly serial profiling and later BVH acceleration for the mesh-heavy creative scenes.

References

[1] *Computer Graphics Rendering—Complete Lecture Notes*. Undated lecture-note compilation, Parts I, III, IV and VI.

[2] *Computer Graphics: Rendering*. Undated lecture, Speaker discussions of ray casting, nearest intersections, hit records and shading.

[3] Peter Shirley. *Ray Tracing in One Weekend*, version 1.54, 2018. Chapters 8 (Metal), 9 (Dielectrics) and 11 (Defocus Blur).